information

MDPI

Article

Sustainable Real-Time NLP with Serverless Parallel Processing on AWS

Chaitanya Kumar Mankala * and Ricardo J. Silva

Computer Science Department, College of Liberal Arts and Science, Villanova, Pennslyvania, PA 19085, USA; ricardo.silva@villanova.edu

* Correspondence: cmankala@villanova.edu; Tel.: +1-484-988-2077

Abstract

This paper proposes a scalable serverless architecture for real-time natural language processing (NLP) on large datasets using Amazon Web Services (AWS). The framework integrates AWS Lambda, Step Functions, and S3 to enable fully parallel sentiment analysis with Transformer-based models such as DistilBERT, RoBERTa, and ClinicalBERT. By containerizing inference workloads and orchestrating parallel execution, the system eliminates the need for dedicated servers while dynamically scaling to workload demand. Experimental evaluation on the IMDb Reviews dataset demonstrates substantial efficiency gains: parallel execution achieved a 6.07× reduction in wall-clock duration, an 81.2% reduction in total computing time and energy consumption, and a 79.1% reduction in variable costs compared to sequential processing. These improvements directly translate into a smaller carbon footprint, highlighting the sustainability benefits of serverless architectures for AI workloads. The findings show that the proposed framework is model-independent and provides consistent advantages across diverse Transformer variants. This work illustrates how cloud-native, event-driven infrastructures can democratize access to large-scale NLP by reducing cost, processing time, and environmental impact while offering a reproducible pathway for real-world research and industrial applications.

Keywords: natural language processing (NLP); serverless computing; AWS lambda; parallel processing; cloud-native architectures; sentiment analysis; energy efficiency; carbon footprint reduction

check for updates

Academic Editor: Katsuhide Fujita

Received: 23 July 2025
Revised: 11 September 2025
Accepted: 3 October 2025
Published: 15 October 2025

Citation: Mankala, C.K.; Silva, R.J. Sustainable Real-Time NLP with Serverless Parallel Processing on AWS. *Information* **2025**, *16*, 903. https://doi.org/10.3390/info16100903

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In today's digital world, the widespread adoption of artificial intelligence (AI), particularly Large Language Models (LLMs) such as BERT and GPT, is driving a sharp increase in energy demand and CO_2 emissions from data centers worldwide [1,2]. Estimates suggest that AI data center clusters could consume more than 1000 terawatt-hours annually by 2030 [3]. Hyperscale data centers—often consuming 100 MW or more, comparable to the needs of a small city—depend on water-intensive cooling systems that add further power requirements [4,5]. These facilities now account for a substantial portion of global greenhouse gas emissions [6]. A single model like GPT-3, for example, requires an estimated 1.29 million kWh of electricity and generates over 550 metric tons of CO_2 emissions during training [7,8]. Beyond environmental concerns, the immense computational demand of modern AI also leads to higher costs and longer processing times, which conventional processing techniques are ill-equipped to manage efficiently [9–11].

While conventional neural networks laid the foundation for AI, deep learning models—particularly those based on the Transformer architecture—have revolutionized the field [12–17]. Models such as BERT and the Generative Pre-trained Transformer (GPT) series learn hierarchical data representations automatically, reducing the need for manual feature engineering [18–20]. However, this depth comes with trade-offs: vastly larger parameter counts, reliance on massive datasets, and dependence on specialized hardware such as GPUs, all of which increase training and inference costs [21,22].

This paper addresses the critical trade-off between the performance of modern LLMs and their high resource consumption. We propose a more efficient and sustainable approach to AI by designing a serverless parallel processing architecture on Amazon Web Services (AWS), aimed at reducing energy usage, cost, and processing time for large-scale workloads. The framework leverages DistilBERT, a Transformer model created through knowledge distillation [23–25], which compresses large "teacher" models like BERT into smaller "student" models with fewer parameters while retaining approximately 97% of the original accuracy [26–28]. DistilBERT's efficiency makes it particularly well-suited for deployment in scalable serverless environments such as AWS Lambda [10]. The framework is validated on a widely used benchmark dataset for sentiment analysis (IMDb Reviews) to ensure reproducibility and comparability with prior studies [29].

To demonstrate model independence, additional experiments are conducted with RoBERTa and ClinicalBERT. Collectively, these evaluations highlight the broader applicability of the proposed architecture across diverse NLP models and domains.

2. Methodology

2.1. Selecting Large Language Model and Dataset

BERT is a large "teacher" model with a multi-layer Transformer encoder, widely used for high-accuracy NLP tasks such as sentiment analysis [30]. DistilBERT is a compact "student" model produced via knowledge distillation [31,32] retaining most of the teacher's performance while reducing parameters and inference cost—an attractive fit for scalable, serverless deployment [33–35]. In this study we use sentiment analysis as a representative, compute-intensive NLP task. Our aim is not to optimize model accuracy but to evaluate how a serverless parallel architecture reduces compute time, cost, and energy usage at scale [36,37]. Well-validated datasets enable reproducible benchmarking. Common options include Amazon Customer Reviews, IMDb, and SST-2. We selected IMDb for its binary labels and established use in benchmarking, providing a robust testbed for parallel processing analysis [38].

2.2. Serverless Parallel Processing Architecture Framework

The core compute layer relies on AWS Lambda to deliver event-driven, autoscaling execution with fine-grained billing (GB·s), enabling cost- and energy-efficient parallelism for inference workloads [39,40]. The framework (Figure 1) integrates four logical zones for clear separation of concerns and reproducibility:

- Data Ingestion and Storage: Amazon S3 serves as the data lake for input text (IMDb subsets) and for storing outputs [41]. Datasets are uploaded and organized for batch processing, and results are written back for downstream analysis.
- Parallel Processing Orchestration: AWS Step Functions coordinates nested parallel execution (e.g., Map/Inline Map), controlling task fan-out/fan-in and error handling [42]. This allows thousands of independent document-level inferences to execute concurrently under policy-controlled limits.

- Deep Learning Inference: Containerized model inference is packaged in Amazon ECR and executed by Lambda (container images), ensuring consistent environments and fast cold-starts with pre-fetched artifacts.
- Output and Analysis: Amazon CloudWatch provides centralized logging/metrics for workflow runs and billed-duration capture; Lambda's serverless monitoring guidance supports traceability and performance tuning. AWS IAM enforces least-privilege access across services (buckets, state machines, functions, and logs).

Figure 1 (architecture overview) and Figure 2 (phase flow) illustrate the complete workflow from local setup to cloud execution and analysis.

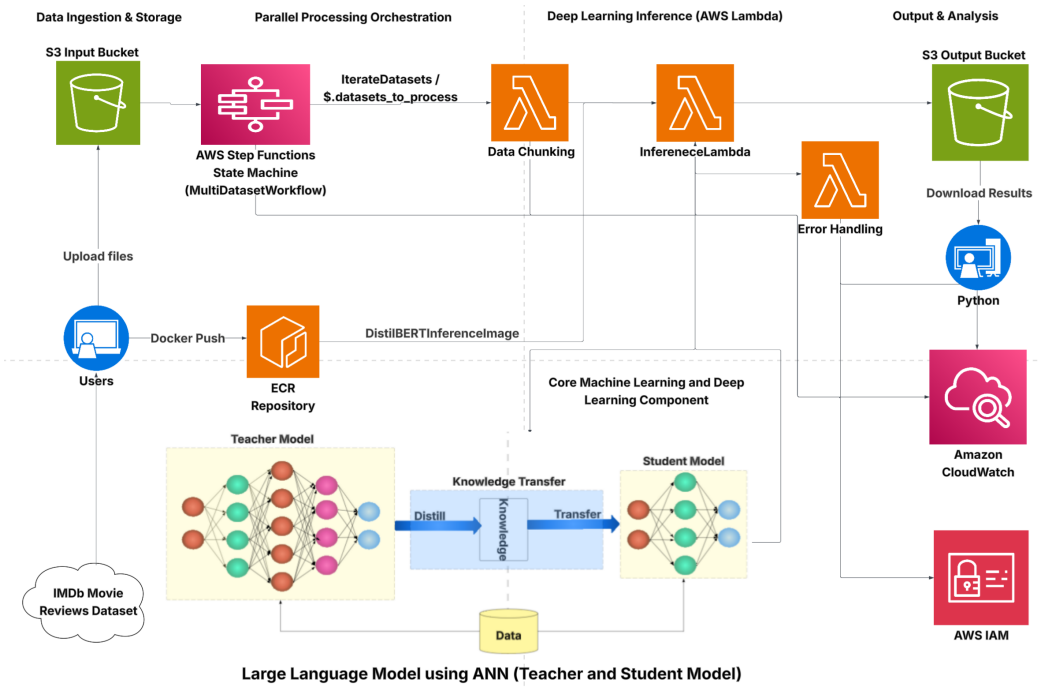


Figure 1. Serverless parallel processing architecture on AWS.

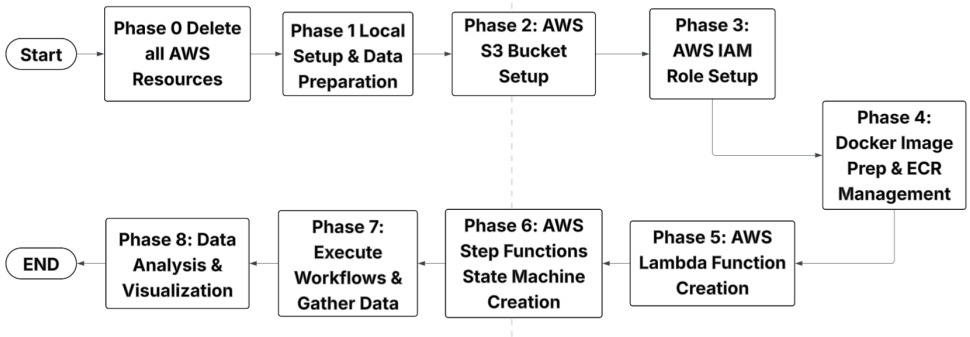


Figure 2. Step-by-step approach using a flow chart representation for the architecture framework.

Local Environment Setup and Data Preparation

The local environment setup included key Python version 3.12 by Tom Sjogren and Tina Sjogren in Bishop, CA, USA on December 1989 dependencies for model development and cloud integration. The Transformers library enabled model interaction [43–47], PyTorch version 2.5.1, by Meta AI on September 2016 in San Jose, CA, USA served as the deep learning framework, boto3 was used for AWS SDK programmatic interaction [48], pandas and openpyxl supported data manipulation, matplotlib and seaborn were used for data visualization [49], wordcloud provided textual insights, and scikit-learn supplied statistical evaluation metrics. The AWS Command Line Interface (CLI) [50] was config-

ured with required credentials, and all executions were performed in the us-east-2 (Ohio) region. The CLI user setup followed least-privilege principles post-deployment, aligning with AWS IAM best practices. Python scripts [51] were used to download the pre-trained `distilbert-base-uncased-finetuned-sst-2-english` model, with local caching implemented to streamline Docker image builds. Granular IAM roles and policies were applied to secure interactions between AWS services, again ensuring least-privilege compliance.

AWS infrastructure was provisioned through CLI to ensure reproducibility and automation. Two S3 buckets were created: one for input data ingestion and another for results storage, with access controls configured to block public access [52,53]. Six distinct subsets of the IMDb Reviews dataset were prepared using Python scripts from the larger dataset. These subsets served as feeder inputs for batch processing. For Docker image preparation and ECR management, a Dockerfile was built on the Python 3.11 Lambda base image. Core dependencies such as NumPy and boto3 (pinned version with precompiled wheels) were included to optimize Lambda compatibility. The built container image was pushed to Amazon ECR for deployment. Three AWS Lambda functions formed the compute backbone of the architecture. `DistilBERTInferenceLambda`, packaged from the Docker image, configured with 3008 MB memory and 15-min timeout, executed sentiment analysis and wrote results directly to S3. `InferenceBatchPreparer`, deployed as a ZIP archive (Python script), configured with 256 MB memory and 5-min timeout, listed S3 files and returned an array of payloads. `InferenceErrorHandler`, deployed as a ZIP archive with 128 MB memory and 30-s timeout, logged details of failed tasks. AWS Step Functions orchestrated the workflow. The state machine was defined using Amazon States Language, with a nested Inline Map: the outer Map iterated over dataset prefixes, and the inner Map concurrently processed files. This was created via AWS CLI [54], providing a robust mechanism for defining complex workflows.

2.3. Custom Dataset Characterization

To systematically evaluate architecture performance, we generated six subsets from the IMDb Reviews dataset [55]. Each subset consisted of 100 text files with predefined sentiment distributions, ensuring diverse workload conditions and balanced benchmarking. These subsets enabled assessment of both processing time and classification performance under controlled sentiment proportions. Table 1 shows the composition of custom IMDb subsets used in experimental evaluation.

Table 1. Composition of custom IMDb subsets used in experimental evaluation.

Dataset Name	Positive Reviews (%)	Negative Reviews (%)	Total Files
`local_imdb_100N_files`	0	100	100
`local_imdb_100P_files`	100	0	100
`local_imdb_55N_45P_files`	45	55	100
`local_imdb_55P_45N_files`	55	45	100
`local_imdb_20P_80N_files`	20	80	100
`local_imdb_80P_20N_files`	80	20	100

Note: All subsets were uploaded to an Amazon S3 input bucket [56], with results stored in a separate output bucket. Subsets enabled controlled experiments comparing sequential (MaxConcurrency: 1) versus parallel (MaxConcurrency: 100) execution in AWS Step Functions.

2.4. Data Analysis and Visualization

AWS provides service usage metrics such as CPU hours, gigabyte-seconds of Lambda execution, and S3 storage volume. These raw measures are translated into estimated energy consumption (kWh) and carbon emissions (kg CO_2e) using the AWS Customer Carbon Footprint Tool (CCFT) [29,57]. This tool accounts for server type, utilization rates, and the Power Usage Effectiveness (PUE) of the hosting data center. Emissions factors, expressed

in kg CO_2e per kWh, are derived from the energy mix of the local grid and are based on standards from the U.S. Environmental Protection Agency (EPA), the Greenhouse Gas (GHG) Protocol, and ISO methodologies. For the US-East-2 (Ohio) region, the following formulas were applied:

- Total Lambda Compute (GB·s)

$$C_{\text{compute}} = \frac{T_{\text{billed_duration(ms)}}}{1000} \times 3.008 \text{ GB}$$

- Estimated Energy Consumption (kWh)

$$E_{\text{energy}} = C_{\text{compute}} \times 1.0 \times 10^{-10} \frac{\text{kWh}}{\text{GB} \cdot \text{s}}$$

- Estimated CO_2 Emissions (kg CO_2e)

$$C_{CO_2} = E_{\text{energy}} \times 4.0 \times 10^{-5} \frac{\text{kg } CO_2\text{e}}{\text{kWh}}$$

- Estimated Variable Cost (USD)

$$C_{\text{variable}} \approx (C_{\text{compute}} \times \$0.0000166667/\text{GB} \cdot \text{s}) + (N_{\text{files}} \times \$0.0000004/\text{S3 GET})$$

These metrics provide a consistent way to evaluate compute efficiency, cost, and environmental impact across experimental conditions.

2.5. Limitations

It is important to note that this study is not an optimization study for DistilBERT's predictive performance. We expect its classification accuracy to be lower than larger transformer models. Instead, the primary goal is to demonstrate the efficiency of the serverless parallel processing framework in reducing processing time, cost, energy consumption, and CO_2 emissions. To further validate model independence, additional experiments were conducted with RoBERTa and ClinicalBERT. Neither model was optimized for sentiment analysis, so their raw predictive performance is expected to be lower than DistilBERT. However, if the proposed architecture is indeed model-agnostic, we should observe similar relative gains from parallel execution regardless of the model deployed.

3. Results

The serverless architecture was evaluated using six IMDb subsets under both sequential execution (MaxConcurrency = 1) and parallel execution (MaxConcurrency = 100). Metrics collected included wall-clock duration, billed compute, estimated energy consumption, CO_2 emissions, and variable cost. Tables 2 and 3 summarize the measured results for 12 distinct workflow executions on the six IMDb subsets under sequential and parallel modes.

To better visualize these results, Figure 3 displays violin plots across key performance and sustainability metrics, confirming that parallel execution significantly improves efficiency.

The classification performance of DistilBERT across the six IMDb subsets is shown in Figure 4. Predicted sentiment distributions align with the ground-truth bias of each dataset, validating the experimental setup.

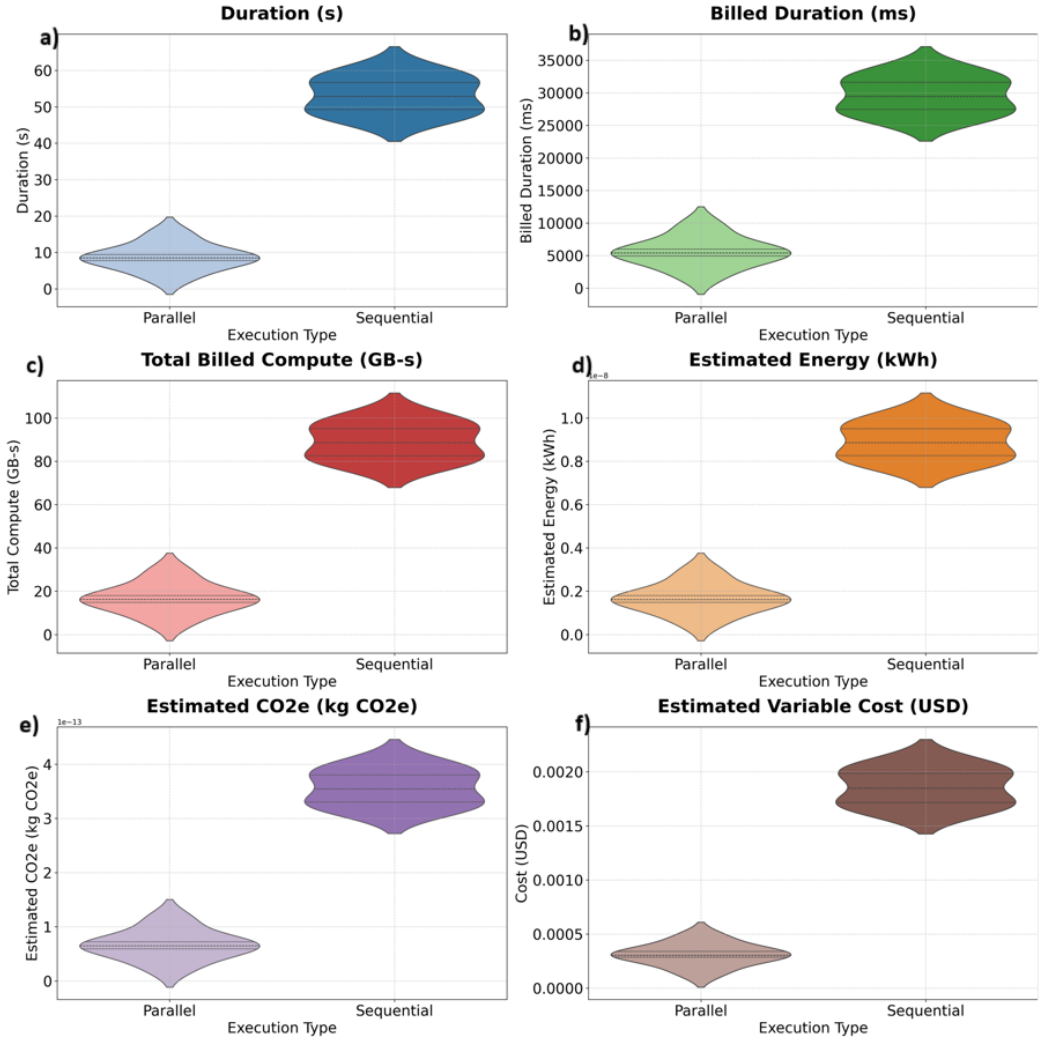


Figure 3. Comparative performance and environmental impact of sequential vs. parallel execution. (**a**) Wall clock duration; (**b**) Lambda billed computation; (**c**) total computation; (**d**) energy consumption; (**e**) CO_2 emissions; (**f**) variable cost.

Table 2. Shows duration, billed duration, and total computation across 12 distinct dataset executions.

Run Name	Type	Time (s)	Billed Time (ms)	Total Compute
100 Neg	Parallel	14.68	9311	28.01 GB-S
100 Neg	Sequential	56.354	31,426	94.54 GB-S
80 Pos/20 Neg	Parallel	9.68	6140	18.47 GB-S
80 Pos/20 Neg	Sequential	59.545	33,205	99.8 GB-S
80 Neg/20 Pos	Parallel	7.646	4850	14.59 GB-S
80 Neg/20 Pos	Sequential	49.339	27,514	82.71 GB-S
55 Neg/45 Pos	Parallel	3.525	2236	6.73 GB-S
55 Neg/45 Pos	Sequential	47.535	26,508	79.69 GB-S
55 Pos/45 Neg	Parallel	8.248	5231	15.73 GB-S
55 Pos/45 Neg	Sequential	49.242	27,460	82.55 GB-S
100 Pos	Parallel	8.774	5565	16.74 GB-S
100 Pos	Sequential	56.847	31,701	95.34 GB-S

To evaluate model independence, the same workflows were repeated with RoBERTa and ClinicalBERT. Despite differences in raw accuracy, parallel execution provided consistent efficiency gains across all models. Figure 5 summarizes time and computation across the threee modles where as Figure 6 summarizes efficiency across energy and CO_2.

These comparisons, reinforcing that the benefits are due to architecture design rather than model choice.

Table 3. Energy, estimated CO_2 emissions, and cost in USD execution results across 12 distinct datasets.

Run Name	Type	Est. Energy (kWh)	Est. CO_2e (kg)	Cost (USD)
100 Neg	Parallel	2.80×10^{-9}	1.12×10^{-13}	0.000467
100 Neg	Sequential	9.45×10^{-9}	3.78×10^{-13}	0.001976
80 Pos/20 Neg	Parallel	1.85×10^{-9}	7.39×10^{-14}	0.000348
80 Pos/20 Neg	Sequential	9.98×10^{-9}	3.99×10^{-13}	0.002058
80 Neg/20 Pos	Parallel	1.46×10^{-9}	5.84×10^{-14}	0.000283
80 Neg/20 Pos	Sequential	8.27×10^{-9}	3.31×10^{-13}	0.001718
55 Neg/45 Pos	Parallel	6.73×10^{-10}	2.69×10^{-14}	0.000152
55 Neg/45 Pos	Sequential	7.97×10^{-9}	3.19×10^{-13}	0.001666
55 Pos/45 Neg	Parallel	1.57×10^{-9}	6.29×10^{-14}	0.000297
55 Pos/45 Neg	Sequential	8.26×10^{-9}	3.30×10^{-13}	0.001715
100 Pos	Parallel	1.67×10^{-9}	6.70×10^{-14}	0.000314
100 Pos	Sequential	9.53×10^{-9}	3.81×10^{-13}	0.00199

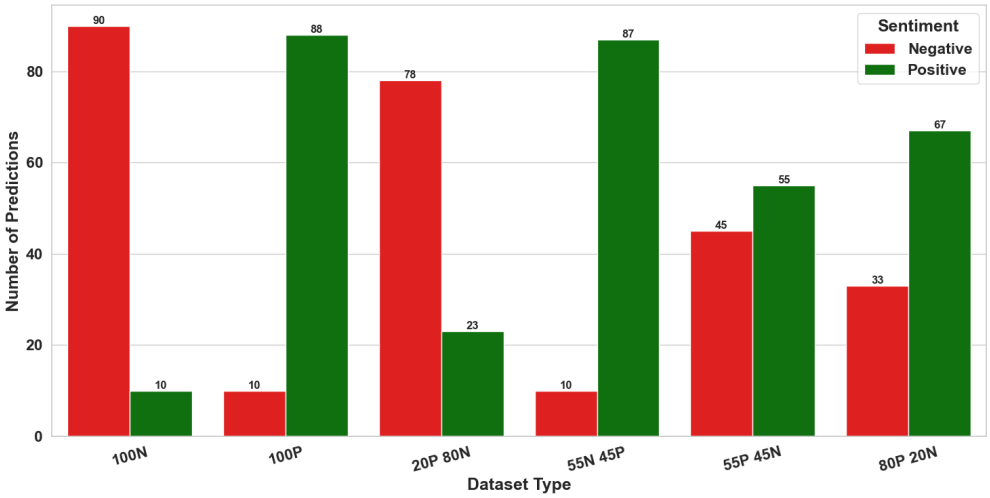


Figure 4. Computed sentiment distribution across six IMDb datasets using DistilBERT.

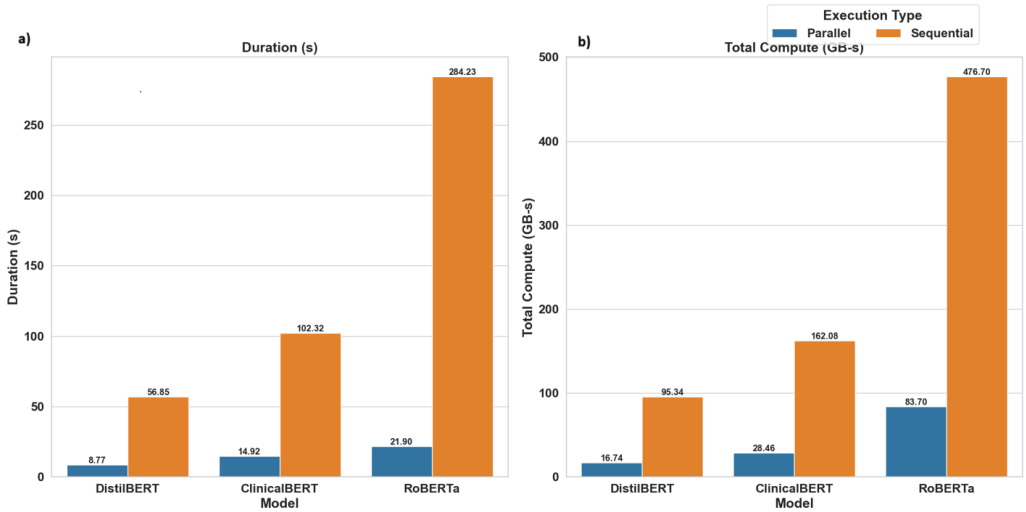


Figure 5. Efficiency gains across multiple Transformer models (DistilBERT, RoBERTa, ClinicalBERT). (**a**) Wall clock duration; (**b**) billed computation.

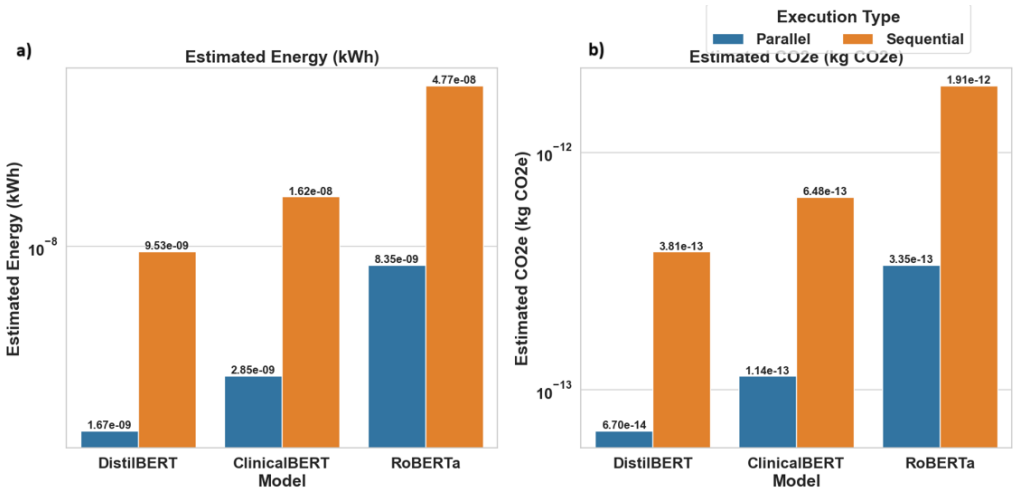


Figure 6. Efficiency gains across multiple Transformer models (DistilBERT, RoBERTa, ClinicalBERT). (**a**) Energy consumption; (**b**) CO_2 emissions.

Scalability Extrapolation

To evaluate scalability, workloads were expanded to 1000 and 10,000 reviews. Using AWS resource usage equations and extrapolation methods [30], we calculated wall clock duration, total compute, energy consumption, CO_2 footprint, and variable cost. Table 4 shows the evidence based calculated performance, cost, and environmental impact at increasing dataset sizes.

Table 4. Calculated performance, cost, and environmental impact at increasing dataset sizes.

Files	Execution	Time (s)	Compute	Energy	CO_2 (kg)	Cost (USD)
100	Parallel	8.76	16.71 GBs	1.67×10^{-9} kWh	6.68×10^{-14}	0.000319
	Sequential	53.14	89.11 GBs	8.91×10^{-9} kWh	3.56×10^{-13}	0.001525
1000	Parallel	87.64	167.12 GBs	1.67×10^{-8} kWh	6.68×10^{-13}	0.003185
	Sequential	531.36	891.05 GBs	8.91×10^{-8} kWh	3.56×10^{-12}	0.015251
10,000	Parallel	876.4	1671.17 GBs	1.67×10^{-7} kWh	6.68×10^{-12}	0.03185
	Sequential	5313.6	8910.50 GBs	8.91×10^{-7} kWh	3.56×10^{-11}	0.15251

Parallel execution shows ∼6× speedup and ∼80% reductions in computation, cost, and CO_2 emissions across scales. Figures 7 and 8 visualizes these scalability trends, clearly showing that parallel execution offers dramatic time and resource savings as dataset size grows.

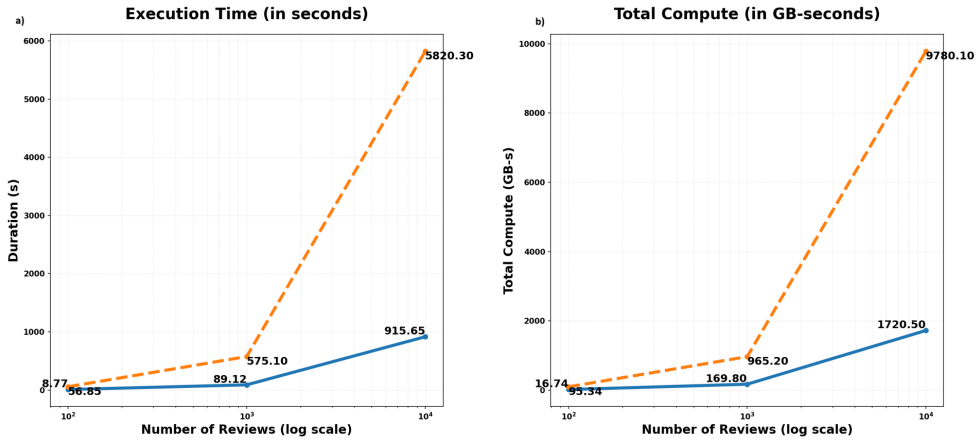


Figure 7. Duration and total computation task performance comparison between sequential and parallel execution across 100, 1000, and 10,000 samples.

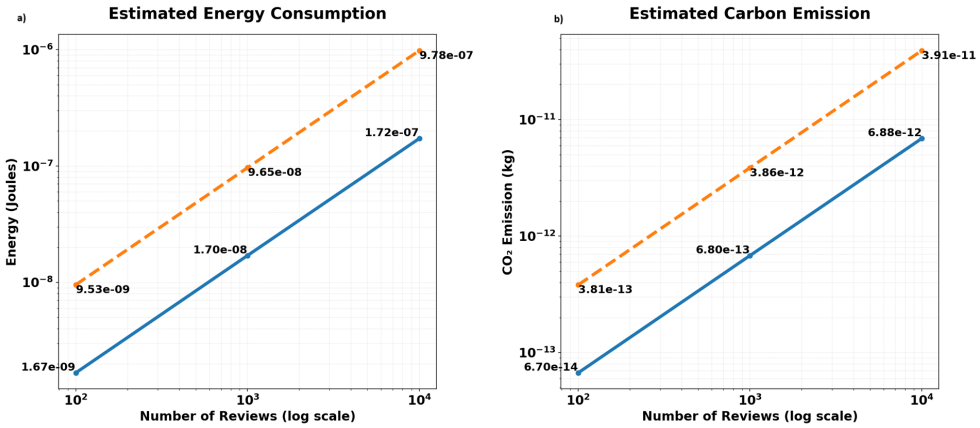


Figure 8. Estimated Energy and estimated CO_2 tasks performance comparison between sequential and parallel execution across 100, 1000, and 10,000 samples.

4. Discussion

The experiments conducted on six IMDb subsets and the scalability extrapolation provide strong evidence that serverless parallel processing offers substantial performance and sustainability advantages for NLP workloads. The most impactful result is the dramatic reduction in wall-clock duration achieved through parallel execution. As shown in Figures 3, 6 and 8, parallel runs consistently completed faster than sequential runs across all datasets and scales. For 100 tasks, parallel execution achieved a ∼6.07× speedup (8.76 s vs. 53.14 s), representing an ∼83.5% reduction in processing time. At scale, even workloads of 1 billion tasks, which are infeasible under sequential execution, become tractable under the parallel framework.

In terms of compute usage, parallel processing proved ∼5.33× more efficient, with an ∼81.2% reduction in billed resources (16.71 GB·s vs. 89.11 GB·s for 100 tasks). This translates directly into cost savings, with parallel execution being ∼4.79× cheaper (∼79.1% reduction in variable cost, e.g., $3185.22 vs. $15,250.86 for 1 billion tasks). These improvements also yield environmental benefits: energy use and CO_2 emissions were both reduced by ∼81.2%, reflecting the efficiency of the serverless parallel architecture [58–61]. Collectively, these results highlight the viability of elastic, serverless infrastructures for scaling AI workloads while supporting corporate sustainability commitments (e.g., Microsoft's carbon-negative goal by 2030) [32]. Beyond quantitative metrics, Figure 9 provides qualitative insight into sentiment classification by visualizing word clouds for reviews predicted as negative (Figure 9a) and positive (Figure 9b). These visualizations highlight frequent terms that align with expected sentiment categories, offering intuitive interpretability of model outputs [62].

To statistically validate differences between sequential and parallel executions, an ANOVA test was performed. Results confirmed significant differences in runtime, compute consumption, and CO_2 emissions across execution types as shown in Figure 10.

Parallel execution consistently demonstrated superior efficiency across scales, offering compounding benefits in cost, compute, and energy reduction [63]. As shown in Figure 11. Extrapolation analysis [64] further demonstrates that efficiency gains persist at increasing workload sizes, underscoring scalability as a defining strength of this approach.

Finally, classification accuracy was analyzed using a confusion matrix. Out of 300 true negative reviews, 177 were correctly classified, while 123 were misclassified as positive (false positives). For the 300 true positive reviews, 212 were correctly classified, while 88 were misclassified as negative (false negatives). Overall accuracy reached 64.83%. However, recall for negative sentiment was only 0.59, and precision for positive sentiment was 0.63. This shows that DistilBERT tended to over-predict positive sentiment, which

could lead to missed detection of critical negative feedback in real-world applications. These findings suggest areas for future refinement, such as retraining with more balanced data or adjusting classification thresholds [65,66].



(**a**) Word cloud of negative reviews across all datasets.

(**b**) Word cloud of positive reviews across all datasets.

Figure 9. (**a**) Visually highlights frequent terms in movie reviews predicted as 'NEGATIVE', while (**b**) visually highlights POSITIVE' predictions. Both provide qualitative insight into sentiment-associated vocabulary, with larger fonts indicating higher frequency.

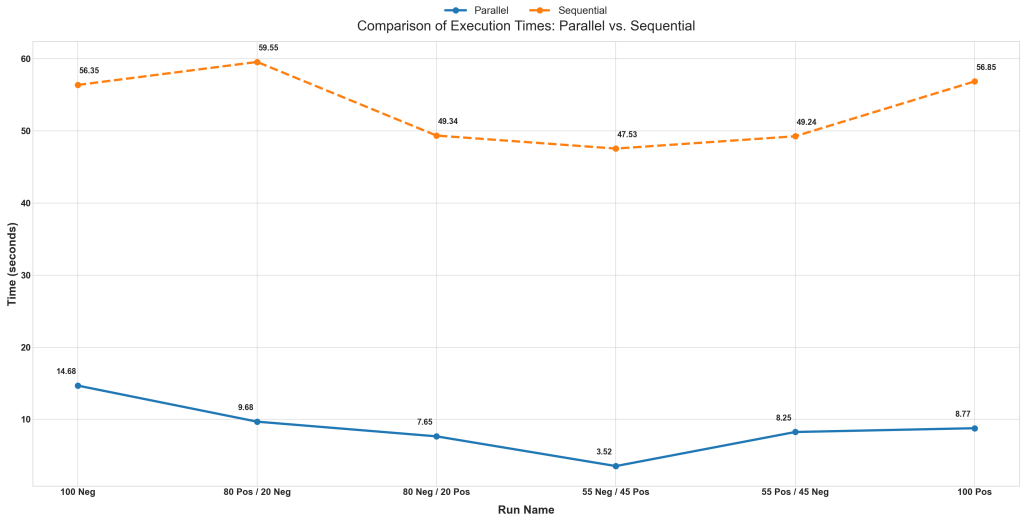


Figure 10. Wall clock duration across dataset mixes (parallel vs. sequential).

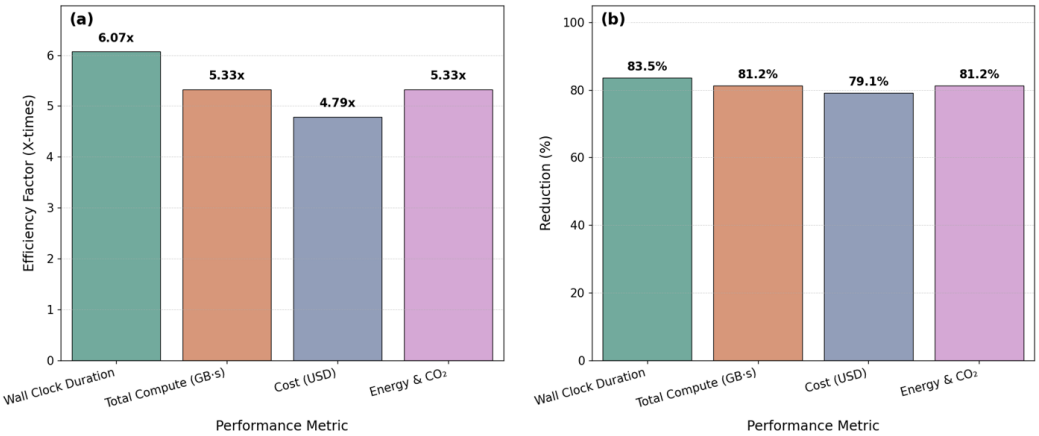


Figure 11. Benefits of parallel processing across execution scales. (**a**) Efficiency factor of parallel vs. sequential processing; (**b**) percentage reduction with parallel processing across duration, computation, cost, energy and CO_2 footprint.

5. Conclusions

This study addressed the environmental and economic challenges of large-scale AI workloads, which are characterized by significant energy consumption, high CO_2 emissions, elevated costs, and prolonged processing durations. We proposed a serverless parallel processing framework on Amazon Web Services (AWS) that leverages DistilBERT, an efficient knowledge-distilled model, as the core inference engine. By exploiting the elasticity of serverless architectures, the framework minimizes infrastructure overhead while scaling efficiently to workload demand. Experimental results demonstrated substantial improvements: a $\sim$6.07$\times$ reduction in wall-clock time, an $\sim$81.2% reduction in total computation and associated energy use, and a $\sim$79.1% reduction in variable costs compared to sequential execution. These findings reaffirm the critical role of cloud-native, event-driven infrastructures in making large-scale deep learning both feasible and sustainable. In addition, the framework successfully performed sentiment analysis with an overall accuracy of 64.83%. Although predictive performance was not the primary focus, diagnostic insights from the confusion matrix revealed areas for refinement, in particular, improving recall for negative sentiment through balanced training data or threshold tuning. Overall, this research validates a scalable and sustainable pathway for deploying AI workloads, demonstrating that advanced computational performance can be achieved responsibly while aligning with organizational carbon-reduction commitments.

6. Limitations and Future Work

While the proposed framework showed clear benefits, several limitations remain. Scalability constraints: AWS Step Functions Inline Map imposes a 256-payload limit, restricting direct use on very large datasets. Similarly, Lambda's default concurrency limit (100 in our environment) constrains full-scale parallelism. Simplified cost model: Estimates included Lambda compute and S3 GET requests but excluded minor charges such as Step Function transitions and long-term S3 storage. Static resource allocation: Experiments fixed Lambda memory at 3008 MB; adaptive tuning could further optimize performance and cost. Model generalization: In this study, DistilBERT, RoBERTa, and ClinicalBERT were deployed without task-specific optimization. As a result, predictive accuracy was limited (an overall accuracy of 64.83% for DistilBERT). This choice was deliberate to isolate the effects of the architecture. Now that the scalability and efficiency of the framework have been demonstrated, future work should focus on fine-tuning models for the target

application domain, which is expected to improve accuracy and reduce misclassification errors. Future research will therefore prioritize:

- Native scalability using a step function distributed map to process datasets with 50,000+ files.
- Cost optimization through AWS Lambda power tuning and more granular total cost of ownership models.
- Enhanced sustainability metrics by integrating AWS-native carbon reporting tools for precise emissions analysis.
- Improved inference diagnostics, including confidence scores and richer evaluation scripts.
- Task-specific model optimization, retraining or fine-tuning models (e.g., DistilBERT) to the target dataset to boost classification accuracy alongside efficiency.
- Robustness testing with large, real-world streaming datasets, exploring integration with AWS Glue, Kinesis, or Data Pipeline.
- Application to new domains, particularly neuroscience, where parallel serverless architectures can accelerate preprocessing, feature extraction, and model inference for detecting early markers of Alzheimer's or dementia.

The framework's event-driven, autoscalable, and cost-efficient design continues to demonstrate strong potential as a foundation for both domain-specific AI and broad industrial applications.

Author Contributions: Conceptualization, R.J.S.; supervision, R.J.S.; validation, R.J.S.; formal analysis, C.K.M.; investigation, C.K.M.; resources, C.K.M.; data curation, C.K.M.; visualization, C.K.M.; writing—original draft preparation, C.K.M.; writing—review and editing, R.J.S. and C.K.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original data presented in the study are openly available in GIT at https://github.com/CMankala/ParallelProcessingUsingServerlessArchitecture/tree/main.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Jones, N. Measuring AI's Energy/Environmental Footprint to Access Impacts. FAS. 2025. Available online: https://fas.org/publication/measuring-and-standardizing-ais-energy-footprint/ (accessed on 4 April 2025).
2. Jones, N. The Growing Energy Demand of Data Centers: Impacts of AI and Cloud Computing. ResearchGate. 2025. Available online: https://www.researchgate.net/publication/388231241_The_Growing_Energy_Demand_of_Data_Centers_Impacts_of_AI_and_Cloud_Computing (accessed on 5 April 2025).
3. Desroches, C.; Chauvin, M.; Ladan, L.; Vateau, C.; Gosset, S.; Cordier, P. Exploring the Sustainable Scaling of AI Dilemma: A Projective Study of Corporations' AI Environmental Impacts. *arXiv* **2025**, arXiv:2501.14334. [CrossRef]
4. Wolters Kluwe. Energy Demands Will Be a Growing Concern for AI Technology. 2025. Available online: https://www.wolterskluwer.com/en/expert-insights/energy-demands-will-be-a-growing-concern-for-ai-technology (accessed on 10 April 2025).
5. Pritchard, R. PREDICTIONS. Digitalisation World. 2024. Available online: https://cdn.digitalisationworld.com/uploads/pdfs/056bca8e0be4e84789c8a98ffdb2fbb2198a2745208f2fbc.pdf (accessed on 20 April 2025).
6. Ariyanti, S.; Suryanegara, M.; Arifin, A.S.; Nurwidya, A.I.; Hayati, N. Trade-Off Between Energy Consumption and Three Configuration Parameters in Artificial Intelligence (AI) Training: Lessons for Environmental Policy. *Sustainability* **2025**, *17*, 5359. [CrossRef]
7. Yu, Z.; Wu, Y.; Deng, Z.; Tang, Y.; Zhang, X.-P. OpenCarbonEval: A Unified Carbon Emission Estimation Framework in Large-Scale AI Models OpenReview. *arXiv* **2024**. [CrossRef]
8. Mondal, S.; Faruk, F.B.; Rajbongshi, D.; Efaz, M.M.K.; Islam, M.M. GEECO: Green Data Centers for Energy Optimization and Carbon Footprint Reduction. *Sustainability* **2023**, *15*, 15249. [CrossRef]
9. Li, Y.; Lin, Y.; Wang, Y.; Ye, K.; Xu, C. Serverless Computing: State-of-the-Art, Challenges and Opportunities. *IEEE Trans. Serv. Comput.* **2022**, *16*, 1522–1539. [CrossRef]

10. Zheng, J. A Large-Scale 12-Lead Electrocardiogram Database for Arrhythmia Study (Version 1.0.0). *PhysioNet* **2022**. [CrossRef]
11. Mankala, C.; Silva, R. Evolutionary Artificial Neuroidal Network Using Serverless Architecture. Available online: https://www.proquest.com/pqdtglobal/docview/3249536286?sourcetype=Dissertations%20&%20Theses/ (accessed on 20 August 2025).
12. Codecademy Team. Understanding Convolutional Neural Network (CNN) Architecture. Codecademy. 2024. Available online: https://www.codecademy.com/article/understanding-convolutional-neural-network-cnn-architecture (accessed on 20 May 2025).
13. IBM. AI vs. Machine Learning vs. Deep Learning vs. Neural Networks. IBM. 2023. Available online: https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks (accessed on 25 May 2025).
14. Coursera. Deep Learning vs. Neural Network: What's the Difference? Coursera. 2025. Available online: https://www.coursera.org/articles/deep-learning-vs-neural-network (accessed on 25 May 2025).
15. H2O.ai. The Difference Between Neural Networking and Deep Learning. H2O.ai. 2024. Available online: https://h2o.ai/wiki/neural-networking-deep-learning/ (accessed on 25 May 2025).
16. Wikipedia. Deep Learning. Wikipedia. 2025. Available online: https://en.wikipedia.org/wiki/Deep_learning (accessed on 25 May 2025).
17. Pure Storage Blog. Deep Learning vs. Neural Networks. Pure Storage Blog. 2022. Available online: https://blog.purestorage.com/purely-educational/deep-learning-vs-neural-networks/ (accessed on 25 May 2025).
18. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv* **2019**. [CrossRef]
19. Analytics Vidhya. Introduction to DistilBERT in Student Model. 2022. Available online: https://www.analyticsvidhya.com/blog/2022/11/introduction-to-distilbert-in-student-model/ (accessed on 25 May 2025).
20. Zilliz Learn. DistilBERT: A Smaller, Faster, and Distilled BERT. 2024. Available online: https://zilliz.com/learn/distilbert-distilled-version-of-bert (accessed on 25 May 2025).
21. Number Analytics. DistilBERT for Efficient NLP. 2025. Available online: https://www.numberanalytics.com/blog/distilbert-efficient-nlp-data-science (accessed on 25 May 2025).
22. AITechTrend. Realizing the Benefits of HuggingFace DistilBERT for NLP Applications. 2023. Available online: https://aitechtrend.com/realizing-the-benefits-of-huggingface-distilbert-for-nlp-applications/ (accessed on 25 May 2025).
23. Analytics Vidhya. A Gentle Introduction to RoBERTa. Analytics Vidhya. 2022. Available online: https://www.analyticsvidhya.com/blog/2022/10/a-gentle-introduction-to-roberta/ (accessed on 25 May 2025).
24. Amazon Web Services. What is the AWS Serverless Application Model (AWS SAM)? Available online: https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html (accessed on 25 May 2025).
25. Singu, S.M. Serverless Data Engineering: Unlocking Efficiency and Scalability in Cloud-Native Architectures. *Artif. Intell. Res. Appl.* **2023**, *3*. https://www.aimlstudies.co.uk/index.php/jaira/article/view/358 (accessed on 25 May 2025).
26. Moody, G.B.; Mark, R.G. The Impact of the MIT-BIH Arrhythmia Database. *IEEE Eng. Med. Biol. Mag.* **2001**, *20*, 45–50. [CrossRef] [PubMed]
27. Irid, S.M.H.; Moussaoui, D.; Hadjila, M.; Azzoug, O. Classification of ecg signals based on mit-bih dataset using bi-lstm model for assisting cardiologists diagnosis. *Trait. Du Signal* **2024**, *41*, 3245. [CrossRef]
28. PhysioNet. PTB Diagnostic ECG Database v1.0.0. 2004. Available online: https://www.physionet.org/physiobank/database/ptbdb/ (accessed on 25 May 2025).
29. Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv* **2019**, arXiv:1907.11692.
30. Alsentzer, E.; Murphy, J.; Boag, W.; Weng, W.H.; Jindi, J.; Naumann, T.; McDermott, M. Publicly Available Clinical BERT Embeddings. *arXiv* **2019**, arXiv:1904.03323. [CrossRef]
31. McAuley, J.; Leskovec, J. Hidden Factors and Hidden Topics: Understanding Consumer Preferences from Reviews. In Proceedings of the 7th ACM Conference on Recommender Systems, Hong Kong, China, 12–16 October 2013; pp. 251–258.
32. Socher, R.; Perelygin, A.; Wu, J.; Chuang, J.; Manning, C.D.; Ng, A.Y.; Potts, C. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, Seattle, WA, USA, 18–21 October 2013; pp. 1631–1642. Available online: https://aclanthology.org/D13-1170.pdf (accessed on 1 June 2025).
33. Serverless Computing—AWS Lambda Pricing—Amazon Web Services. AWS. Available online: https://aws.amazon.com/lambda/pricing/ (accessed on 1 June 2025).
34. What Is Amazon S3?—Amazon Simple Storage Service. AWS. Available online: https://aws.amazon.com/s3/ (accessed on 1 June 2025).
35. Use Amazon S3 as a Data Lake—Amazon S3. AWS. Available online: https://docs.aws.amazon.com/AmazonS3/latest/userguide/data-lake-s3.html (accessed on 1 June 2025).

36. AWS Step Functions—Workflow Orchestration. AWS. Available online: https://aws.amazon.com/step-functions/ (accessed on 1 June 2025).
37. What Is AWS Step Functions? How It Works and Use Cases. Datadog. Available online: https://www.datadoghq.com/knowledge-center/aws-step-functions/ (accessed on 11 June 2025).
38. What Is Amazon Elastic Container Registry?—Amazon ECR. AWS. Available online: https://aws.amazon.com/ecr/ (accessed on 11 June 2025).
39. Deploying Container Images to AWS Lambda—AWS Lambda. AWS. Available online: https://docs.aws.amazon.com/lambda/latest/dg/images-create.html (accessed on 11 June 2025).
40. What Is Amazon CloudWatch?—Amazon CloudWatch. AWS. Available online: https://aws.amazon.com/cloudwatch/ (accessed on 11 June 2025).
41. Monitoring and Logging for Serverless Applications—AWS Lambda. AWS. Available online: https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatch.html (accessed on 11 June 2025).
42. What Is IAM?—AWS Identity and Access Management. AWS. Available online: https://aws.amazon.com/iam/ (accessed on 11 June 2025).
43. PyTorch. PyTorch.org. Available online: https://pytorch.org/ (accessed on 11 June 2025).
44. Boto3 Documentation. Available online: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html (accessed on 11 June 2025).
45. pandas—Python Data Analysis Library. pandas.pydata.org. Available online: https://pandas.pydata.org/ (accessed on 11 June 2025).
46. Matplotlib. Matplotlib.org. Available online: https://matplotlib.org/ (accessed on 11 June 2025).
47. Seaborn: Statistical Data Visualization. Seaborn.pydata.org. Available online: https://seaborn.pydata.org/ (accessed on 11 June 2025).
48. wordcloud · PyPI. PyPI.org. Available online: https://pypi.org/project/wordcloud/ (accessed on 11 June 2025).
49. scikit-learn: Machine Learning in Python. scikit-learn.org. Available online: https://scikit-learn.org/stable/ (accessed on 11 June 2025).
50. AWS Command Line Interface. AWS. Available online: https://aws.amazon.com/cli/ (accessed on 11 June 2025).
51. NumPy. NumPy.org. Available online: https://numpy.org/ (accessed on 11 June 2025).
52. AWS Customer Carbon Footprint Tool. AWS. Available online: https://aws.amazon.com/aws-cost-management/aws-customer-carbon-footprint-tool/ (accessed on 11 June 2025).
53. Greenhouse Gas Protocol—A Corporate Accounting and Reporting Standard. World Resources Institute and WBCSD. Available online: https://ghgprotocol.org/ (accessed on 11 June 2025).
54. *ISO 14064-1:2018*; Greenhouse Gases—Part 1: Specification with Guidance at the Organization Level for Quantification and Reporting of Greenhouse Gas Emissions and Removals. International Organization for Standardization: Geneva, Switzerland, 2018.
55. ANOVA—Analysis of Variance. Investopedia. Available online: https://www.investopedia.com/terms/a/anova.asp (accessed on 11 June 2025).
56. Extrapolation—Definition and Applications. Investopedia. Financial Analysis. Available online: https://www.investopedia.com/terms/f/financial-analysis.asp/ (accessed on 11 June 2025).
57. Microsoft. Microsoft to be Carbon Negative by 2030. Microsoft News. Available online: https://blogs.microsoft.com/blog/2020/01/16/microsoft-will-be-carbon-negative-by-2030/ (accessed on 21 June 2025).
58. Hinton, G.; Vinyals, O.; Dean, J. Distilling the knowledge in a neural network. *arXiv* **2015**, arXiv:1503.02531. [CrossRef]
59. Goldberger, A.; Amaral, L.; Glass, L.; Hausdorff, J.; Ivanov, P.C.; Mark, R.; Mietus, J.E.; Moody, G.B.; Peng, C.K.; Stanley, H.E. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. **2000**, *101*, e215–e220. Available online: https://physionet.org/content/incartdb/1.0.0/ (accessed on 21 June 2025).
60. Jiang, F. Artificial Intelligence in Healthcare: Past, Present, and Future. *Stroke Vasc. Neurol.* **2017**, *2*, 230–243. [CrossRef] [PubMed]
61. Medina-Avelino, J.; Silva-Bustillos, R.; Holgado-Terriza, J.A. Are Wearable ECG Devices Ready for Hospital at Home Application? *Sensors* **2025**, *25*, 2982. [CrossRef] [PubMed]
62. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2–7 June 2019; pp. 4171–4186. Available online: https://aclanthology.org/N19-1423.pdf (accessed on 21 June 2025).
63. imdb_reviews TensorFlow Datasets. 2024. Available online: https://www.tensorflow.org/datasets/catalog/imdb_reviews (accessed on 21 June 2025).
64. IMDB Movie Review Sentiment Classification Dataset. Keras. Available online: https://keras.io/api/datasets/imdb/ (accessed on 21 June 2025).

65. Brownlee, J. What is a Confusion Matrix in Machine Learning. Machine Learning Mastery. 2020. Available online: https://machinelearningmastery.com/confusion-matrix-machine-learning/ (accessed on 21 June 2025).
66. Narkhede, S.K. Understanding Confusion Matrix. Towards Data Science. 2018. Available online: https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62 (accessed on 21 June 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.